
Manual Técnico de Replicabilidad

ETL de la Base Maestra Sisbén IV — Antioquia

etl_sisben_definitivo.py — Versión 3 (Chunk Mode)

Coordinación Departamental del Sisbén

Gobernación de Antioquia

vvanegasa@antioquia.gov.co

Medellín, abril de 2026

Propósito de este documento. Este manual explica, paso a paso y línea por línea, el proceso de transformación de datos que diseñé para procesar la base de datos del Sisbén IV del Departamento de Antioquia. Mi objetivo es que cualquier analista que llegue a este cargo pueda replicarlo exactamente, entenderlo completamente y adaptarlo si el DNP entrega nuevas versiones del archivo. Está escrito en un lenguaje natural y técnico a la vez, sin asumir conocimientos avanzados de programación, pero sin omitir la rigurosidad que el proceso requiere.

Índice

1. Contexto y Punto de Partida	3
1.1. ¿Qué es la base de datos del Sisbén IV?	3
1.2. El problema original: tamaño y complejidad	3
1.3. La solución: migración a Parquet con ETL pre-calculado	3
2. Script Definitivo: etl_sisben_definitivo.py v3	4
2.1. Identificación del script	4
2.2. Prerrequisitos: qué necesita tener instalado	4
3. Arquitectura del Script: La Lógica de Procesamiento por Lotes	5
3.1. ¿Por qué procesamiento por lotes (<i>chunks</i>)?	5
3.2. Flujo general del proceso	5
4. Explicación Línea por Línea	6
4.1. Bloque 1 — Encabezado y configuración inicial (líneas 1–41)	6

4.2.	Bloque 2 — La función <code>to_int</code> (líneas 47–49)	6
4.3.	Bloque 3 — La función principal <code>calcular_columnas</code> (líneas 52–290) . . .	7
4.3.1.	3.1 Normalización de nombres de columnas (líneas 58–69)	7
4.3.2.	3.2 Forzado de tipos en columnas base (líneas 72–143)	8
4.4.	Las 26 Transformaciones DAX Llevadas a Python	8
4.4.1.	Columna 1: <code>orden_clasificacion</code> (líneas 150–155)	9
4.4.2.	Columna 2: <code>Tipo_Vivienda_txt</code> (línea 160)	9
4.4.3.	Columna 3: <code>Material de las Paredes</code> (líneas 163–169)	9
4.4.4.	Columnas 4–8: Servicios Públicos en texto (líneas 178–183)	10
4.4.5.	Columna 9: <code>Clase_txt</code> (línea 187)	10
4.4.6.	Columna 10: <code>Sanitario_txt</code> (líneas 190–194)	10
4.4.7.	Columna 11: <code>Privaciones_Vivienda</code> (líneas 196–202)	10
4.4.8.	Columna 12: <code>Nivel_Calidad_Vivienda</code> (líneas 204–210)	11
4.4.9.	Columna 13: <code>Zona</code> (línea 213)	11
4.4.10.	Columna 14: <code>Es_Jefe</code> (línea 216)	11
4.4.11.	Columnas 15–16: <code>Sexo_Jefe</code> y <code>Sexo_Jefe_Label</code> (líneas 218–225) .	11
4.4.12.	Columna 17: <code>Personas_por_Cuarto</code> (líneas 227–230)	12
4.4.13.	Columna 18: <code>Cat_Hacinamiento</code> (líneas 232–237)	12
4.4.14.	Columna 19: <code>Material del Piso</code> (líneas 240–248)	12
4.4.15.	Columna 20: <code>Orden_Paredes</code> (línea 251)	13
4.4.16.	Columna 21: <code>Tipo_Ocupacion_txt</code> (líneas 253–258)	13
4.4.17.	Columna 22: <code>Rango_Edad</code> (líneas 260–274)	13
4.4.18.	Columna 23: <code>Orden_Edad</code> (líneas 277–283)	13
4.4.19.	Columna 24: <code>Regimen_Salud_Limpio</code> (líneas 285–288)	13
5.	Procesamiento por Chunks: El Motor del ETL	14
5.1.	Inicialización del lector de Parquet (líneas 310–324)	14
5.2.	El ciclo principal (líneas 326–368)	14
6.	Reemplazo Atómico y Verificación Final	15
6.1.	Reemplazo del archivo original (líneas 380–384)	15
6.2.	Verificación automática de tipos (líneas 403–441)	15
7.	Cómo Ejecutar el Script	16
7.1.	Pasos de ejecución	16
7.2.	¿Cuándo se debe volver a ejecutar?	16
8.	Integración con Power BI	17
9.	Resumen de Columnas Generadas por el ETL	17
10.	Notas Técnicas y Advertencias	18

1. Contexto y Punto de Partida

1.1. ¿Qué es la base de datos del Sisbén IV?

El Sistema de Identificación de Potenciales Beneficiarios de Programas Sociales (Sisbén IV) es la herramienta oficial del Departamento Nacional de Planeación (DNP) de Colombia para clasificar a la población según su condición socioeconómica. Para el Departamento de Antioquia, esta base contiene aproximadamente **25 millones de registros individuales**, correspondientes a personas encuestadas en los 125 municipios del Departamento.

El DNP entrega periódicamente a cada Coordinación Departamental un archivo plano con la información anonimizada de todos los registros. Este archivo es la fuente primaria de todas las cifras de cobertura, pobreza y focalización que produce la Coordinación.

1.2. El problema original: tamaño y complejidad

La base de datos en su formato original (archivo delimitado por comas o formato propio del DNP) tiene dos problemas fundamentales para el análisis moderno:

1. **Tamaño:** Con 25 millones de filas y más de 130 columnas, el archivo supera los 10 gigabytes en formato CSV. Herramientas como Excel lo hacen imposible de abrir. Power BI lo puede importar directamente pero con un costo enorme en memoria RAM y tiempo de cálculo.

1.3. La solución: migración a Parquet con ETL pre-calculado

Decidí resolver ambos problemas de una sola vez: migrar la base de datos al formato **Apache Parquet** (un formato de almacenamiento columnar, comprimido y extremadamente eficiente para análisis) y, durante ese proceso de conversión, **calcular de antemano todas las transformaciones** que antes hacía Power BI en tiempo real.

El resultado es un único archivo:

```
D:\VVANEGASA\My Documents\sisben_master_final.parquet
```

Este archivo contiene la base completa de Antioquia con todas las columnas analíticas ya resueltas, comprimida en formato *Snappy*, lista para ser consultada por Power BI prácticamente sin costo computacional adicional.

2. Script Definitivo: etl_sisben_definitivo.py v3

2.1. Identificación del script

El script que documento en este manual es la **versión 3 (v3) del ETL**, que denominé como la versión en modo *Chunk* (procesamiento por lotes). Esta versión reemplaza definitivamente a las versiones anteriores (v1 y v2) por las razones de eficiencia que explico más adelante.

- **Nombre del archivo:** etl_sisben_definitivo.py
- **Ubicación:** D:\VVANEGASA\Desktop\antigravity_sisben\
- **Extensión:** .py (Python 3.10 o superior)
- **Líneas de código:** 450
- **Ejecución estimada:** Entre 8 y 20 minutos dependiendo del hardware

2.2. Prerrequisitos: qué necesita tener instalado

Para ejecutar este script, la persona que lo replique necesita tener instalado Python y las siguientes librerías. Todas se instalan desde la terminal con un solo comando:

```
1 pip install pandas numpy pyarrow
```

Listing 1: Instalación de dependencias

La descripción de cada librería y su rol en el proceso:

Librería	Versión mínima	Rol en el ETL
pandas	1.5.0	Motor principal para manipular filas y columnas. Lo uso para aplicar las transformaciones DAX traducidas a Python.
numpy	1.23.0	Operaciones numéricas vectorizadas, especialmente para los <code>np.select</code> (equivalente a múltiples IF anidados de DAX).
pyarrow	10.0.0	Lectura y escritura del formato Parquet en modo <i>streaming</i> (por lotes). Sin esta librería el script no puede ni leer el archivo fuente ni escribir el resultado.
gc	Estándar	Recolector de basura de Python. Lo uso explícitamente para liberar la memoria de cada lote al terminar de procesarlo.
time	Estándar	Cronómetro interno para medir cuánto tarda cada lote y proyectar el tiempo restante.

Librería	Versión mínima	Rol en el ETL
sys, os	Estándar	Manejo del sistema de archivos (renombrar, eliminar el temporal) y configuración de la consola.

3. Arquitectura del Script: La Lógica de Procesamiento por Lotes

3.1. ¿Por qué procesamiento por lotes (*chunks*)?

Las versiones v1 y v2 del ETL intentaban cargar los 25 millones de filas completos en la memoria RAM del computador de una sola vez. En un equipo con 16 GB de RAM, esto es imposible porque el DataFrame de pandas ocupaba entre 18 y 22 GB al incluir todas las columnas. El computador se quedaba sin memoria a mitad del proceso y el script fallaba.

La versión v3 resuelve esto con una técnica estándar en ingeniería de datos: en lugar de leer todo el archivo de una vez, lo leo en **lotes de 150.000 filas**. A cada lote le aplico todas las transformaciones, lo grabo en el archivo de salida, y luego *descarto completamente ese lote de la memoria* antes de leer el siguiente. En el momento más exigente, el proceso nunca supera los 2 GB de RAM, lo que lo hace perfectamente ejecutable en cualquier computador de oficina moderno.

Analogía práctica. Imagínese que tiene que transcribir un libro de 1.000 páginas a máquina. La estrategia v1 intentaba memorizar las 1.000 páginas antes de empezar a escribir. La estrategia v3 lee 15 páginas, las transcribe, las olvida, y pasa a las 15 siguientes. El resultado final es exactamente el mismo, pero el segundo método es físicamente posible.

3.2. Flujo general del proceso

El script sigue exactamente cuatro etapas que se imprimen en consola para que el operador pueda hacer seguimiento en tiempo real:

1. **[Etapa 1]** Abrir el archivo Parquet fuente y leer sus metadatos (número de filas, grupos).
2. **[Etapa 2]** Iterar por cada lote: convertir a pandas, aplicar 26 transformaciones, ordenar columnas, escribir al archivo temporal.
3. **[Etapa 3]** Cerrar el escritor y reportar el total de chunks procesados.
4. **[Etapa 4]** Reemplazar el archivo original por el nuevo (eliminar el antiguo y renombrar el temporal).

4. Explicación Línea por Línea

4.1. Bloque 1 — Encabezado y configuración inicial (líneas 1–41)

```
18 import sys
19 import os
20 import gc
21 import time
22
23 import pandas as pd
24 import numpy as np
25 import pyarrow as pa
26 import pyarrow.parquet as pq
27
28 sys.stdout.reconfigure(encoding="utf-8")
```

Listing 2: Configuración inicial del script

Primero cargo todas las librerías que voy a necesitar. La línea `sys.stdout.reconfigure(encoding=ütf-8)` es importante: le dice a Python que use el juego de caracteres UTF-8 para imprimir en la consola de Windows. Sin esto, los caracteres especiales del español (tildes, eñes) salen como símbolos extraños en la pantalla.

```
32 PARQUET_IN      = r"D:\VVANEGASA\My Documents\sisben_master.parquet"
33 PARQUET_OUT     = r"D:\VVANEGASA\My Documents\sisben_master_v3.parquet"
34 PARQUET_FINAL   = r"D:\VVANEGASA\My Documents\sisben_master.parquet"
35
36 CHUNK_ROWS     = 150_000
```

Listing 3: Rutas de entrada y salida

Aquí defino tres rutas con propósitos distintos:

- **PARQUET_IN**: El archivo original que voy a leer. Al comienzo del proceso, este es el Parquet que tenía sin las columnas calculadas.
- **PARQUET_OUT**: Un archivo **temporal** donde escribo el resultado mientras proceso. Lo llamo `_v3` para diferenciarlo. Si el proceso falla a mitad, el original (**PARQUET_IN**) no se toca — solo el temporal queda inconsistente y puedo borrarlo sin perder nada.
- **PARQUET_FINAL**: La ruta del destino definitivo. Al final del proceso, borro el original y renombro el temporal a este nombre, haciendo un reemplazo atómico.
- **CHUNK_ROWS = 150_000**: Cada lote tendrá máximo 150.000 filas. Probé con 500.000 y el computador se ponía lento. Con 100.000 el proceso era demasiado lento por la sobrecarga de tantas iteraciones. 150.000 fue el balance perfecto.

4.2. Bloque 2 — La función `to_int` (líneas 47–49)

```

47 def to_int(series):
48     """Int seguro desde cualquier dtype; NaN -> pd.NA (Int64 nullable)."""
49     return pd.to_numeric(series, errors="coerce").astype("Int64")

```

Listing 4: Helper de conversión segura a entero

Esta pequeña función es fundamental. La base del DNP tiene muchas columnas numéricas que llegaron como texto (string) debido a cómo el archivo fue generado. En DAX de Power BI, las conversiones de tipo son automáticas. En Python, si intento comparar `"1" = 1`, el resultado es `False` y todas mis transformaciones fallan silenciosamente.

La función `to_int` resuelve esto de manera segura:

- `pd.to_numeric(series, errors="coerce")`: Convierte cualquier valor al número que representa. Si algo no puede convertirse (por ejemplo, la cadena "NA" o un campo vacío), lo convierte automáticamente a NaN en lugar de lanzar un error.
- `.astype("Int64")`: Usa el tipo **Int64 con N mayúscula** de pandas (el tipo entero *nullable*). A diferencia del `int64` estándar de NumPy, este tipo puede contener valores nulos (`pd.NA`) sin problema. Esto es crítico porque en ciencias sociales los datos siempre tienen campos vacíos.

4.3. Bloque 3 — La función principal `calcular_columnas` (líneas 52–290)

Esta es la función central del ETL. Recibe un fragmento (*chunk*) del DataFrame como entrada y devuelve ese mismo fragmento con todas las columnas analíticas añadidas. La diseñé para que sea un espejo exacto de las columnas calculadas DAX que existían en el modelo de Power BI.

4.3.1. 3.1 Normalización de nombres de columnas (líneas 58–69)

```

58 rename_map = {}
59 for col in df.columns:
60     if col.lower().replace("?", "n").startswith("ind_discap_ba") and "
61     arse" in col.lower():
62         if col != "ind_discap_banarse":
63             rename_map[col] = "ind_discap_banarse"
64         if col.lower().startswith("tip_cuidado_ni") and "os" in col.lower():
65             if col != "tip_cuidado_ninos":
66                 rename_map[col] = "tip_cuidado_ninos"
67         if col.lower() == "num_evento_vendaval" and col != "
68         Num_evento_vendaval":
69             rename_map[col] = "Num_evento_vendaval"
70 if rename_map:
71     df.rename(columns=rename_map, inplace=True)

```

Listing 5: Corrección de nombres de columnas con caracteres especiales

El archivo del DNP tiene un problema histórico: algunas columnas contienen caracteres con tilde o ñe en su nombre (`ind_discap_bañarse`, `tip_cuidado_niños`). Dependiendo de cómo se guardó el Parquet original, estos nombres pueden aparecer con el carácter correcto o con un símbolo roto (?). Esta normalización detecta ambas formas y las unifica al nombre canónico correcto. `Num_evento_vendaval` también tiene una particularidad: en versiones anteriores del archivo estaba en minúsculas y en otras con la N mayúscula — lo estandarizo aquí.

4.3.2. 3.2 Forzado de tipos en columnas base (líneas 72–143)

Esta sección define dos listas y recorre todos los nombres:

- **COLS_INT64:** Una lista de aproximadamente 130 nombres de columnas que deben ser enteros. Incluye todos los indicadores binarios (1/0), los valores de gasto, los códigos de tipo (vivienda, pared, piso, etc.), los índices IPM (I1 a I15) y el identificador de hogar.
- **COLS_STR:** Columnas que deben permanecer como texto: el código del municipio, la clasificación del grupo Sisbén, la fecha de digitación, la marca DNP, el estado EOS y el nombre del archivo fuente.

El código para forzar los tipos es:

```
138 for col in COLS_INT64:
139     if col in df.columns:
140         df[col] = pd.to_numeric(df[col], errors="coerce").astype("Int64")
141
142 for col in COLS_STR:
143     if col in df.columns:
144         df[col] = df[col].astype(str).replace("nan", pd.NA)
```

Listing 6: Aplicación masiva de tipos

Nótese el `if col in df.columns`: siempre verifico que la columna exista antes de intentar modificarla. Esto hace que el script sea robusto ante futuras versiones del archivo del DNP que puedan tener columnas distintas.

Nota crítica sobre estado y marca. Estas dos columnas se mantienen deliberadamente como **texto (String)**. Aunque contienen números (0, 50, 60, 63, etc.), los dejé como texto porque Power BI los usará en filtros de pertenencia a conjuntos (IN {"0", "60"}). Si los convierto a número, las fórmulas DAX de las tres matrices de calidad deben usar comparaciones numéricas en lugar de de texto. La decisión de mantenerlos como texto fue intencional y ya está reflejada en las medidas de las Matrices 1, 2 y 3.

4.4. Las 26 Transformaciones DAX Llevadas a Python

A continuación explico cada columna calculada que añade el ETL, replicando exactamente la lógica DAX que tenía el modelo de Power BI antes de la migración.

4.4.1. Columna 1: orden_clasificacion (líneas 150–155)

```
150 grupo = df["Clasificacion"].astype(str).str[0].where(  
151     df["Clasificacion"].notna() & (df["Clasificacion"].astype(str) != "  
    nan"), other=""  
152 )  
153 df["orden_clasificacion"] = grupo.map(  
154     {"A": 1, "B": 2, "C": 3, "D": 4}  
155 ).fillna(99).astype("Int64")
```

Listing 7: Orden numérico para clasificación Sisbén

¿Por qué existe esta columna? La columna **Clasificacion** de la base original contiene cadenas como `^1`, `B2`, `Ç3`, etc. Para ordenar correctamente los gráficos y la pirámide poblacional en Power BI, necesito un número que permita ordenar: A=1, B=2, C=3, D=4. Los registros sin clasificación reciben el valor 99 para que queden al final de cualquier ordenamiento.

Extraigo solo el primer caracter de la cadena (`.str[0]`) porque la letra del grupo siempre es el primer caracter, independientemente del número que siga.

4.4.2. Columna 2: Tipo_Vivienda_txt (línea 160)

Traduce el código numérico del tipo de vivienda a su etiqueta oficial del DNP:

Código	Etiqueta
1	Casa
2	Apartamento
3	Cuarto
4	Otro tipo de unidad de vivienda
5	Vivienda indígena
Cualquier otro	Sin información

4.4.3. Columna 3: Material de las Paredes (líneas 163–169)

Traduce `tip_mat_paredes` a etiquetas. Esta columna alimenta directamente la matriz de vivienda del tablero de Power BI:

Código	Etiqueta
0	Sin paredes
1	Bloque, ladrillo, piedra, madera pulida
2	Tapia pisada o adobe
3	Bahareque
4	Material prefabricado
5	Madera burda, tabla o tablón
6	Guadua, caña u otro vegetal

4.4.4. Columnas 4–8: Servicios Públicos en texto (líneas 178–183)

Para cada uno de los cinco servicios (energía, acueducto, alcantarillado, gas y recolección de basuras), el código 1 indica que el hogar *sí tiene* el servicio y el código 2 indica que *no lo tiene*. Genero una columna de texto para cada uno:

```

178 _srv_map = {1: "Si tiene servicio", 2: "No tiene servicio"}
179 df["Energia_txt"] = to_int(df["ind_tiene_energia"]).map(_srv_map)
    .fillna("Sin informacion")
180 df["Acueducto_txt"] = to_int(df["ind_tiene_acueducto"]).map(
    _srv_map).fillna("Sin informacion")
181 df["Alcantarillado_txt"] = to_int(df["ind_tiene_alcantarillado"]).map(
    _srv_map).fillna("Sin informacion")
182 df["Gas_txt"] = to_int(df["ind_tiene_gas"]).map(_srv_map).
    fillna("Sin informacion")
183 df["Recoleccion_txt"] = to_int(df["ind_tiene_recoleccion"]).map(
    _srv_map).fillna("Sin informacion")

```

Listing 8: Servicios públicos en texto

4.4.5. Columna 9: Clase_txt (línea 187)

Convierte Cod_clase a texto urbano/rural. Tanto el código 1 (cabecera municipal) como el código 2 (centros poblados) se consideran **urbano** en este análisis. Solo el código 3 (rural disperso) se clasifica como **rural**.

4.4.6. Columna 10: Sanitario_txt (líneas 190–194)

Traduce el tipo de unidad sanitaria a texto descriptivo, según la codificación oficial del DNP.

4.4.7. Columna 11: Privaciones_Vivienda (líneas 196–202)

```

196 for _c in ["I11", "I12", "I13", "I14", "I15"]:
197     df[_c] = to_int(df[_c])
198 df["Privaciones_Vivienda"] = (
199     df["I11"].fillna(0) + df["I12"].fillna(0) + df["I13"].fillna(0) +
200     df["I14"].fillna(0) + df["I15"].fillna(0)
201 ).astype("Int64")

```

Listing 9: Suma de privaciones de vivienda del IPM

¿Qué es esto? Los indicadores del Índice de Pobreza Multidimensional (IPM) se agrupan en cinco dimensiones: Educación (I1–I6), Trabajo (I7–I8), Salud (I9–I10) y Vivienda/Servicios (I11–I15). Esta columna suma únicamente los indicadores de la dimensión de vivienda para cada persona. Un valor de 0 significa *sin privaciones en vivienda*; un valor de 5 significa que el hogar tiene **todas** las privaciones de vivienda posibles (fuente de agua inadecuada, eliminación de excretas incorrecta, materiales de pisos inadecuados, materiales de paredes inadecuados y hacinamiento crítico).

4.4.8. Columna 12: Nivel_Calidad_Vivienda (líneas 204–210)

```

204 _pv = df["Privaciones_Vivienda"]
205 df["Nivel_Calidad_Vivienda"] = np.where(
206     _pv == 0, "Sin privaciones",
207     np.where(_pv <= 2, "Privacion moderada",
208     np.where(_pv >= 3, "Privacion alta", "Sin informacion"))
209 )

```

Listing 10: Categorización de la calidad de vivienda

Convierte el número de privaciones en una categoría ordinal descriptiva. Uso `np.where` porque es el equivalente vectorizado del IF anidado de DAX, y funciona sobre millones de filas en milisegundos.

4.4.9. Columna 13: Zona (línea 213)

Es idéntica a `Clase_txt` en su lógica. La mantengo como columna separada porque en algunos visuales del tablero la llamo por este alias.

4.4.10. Columna 14: Es_Jefe (línea 216)

```

216 df["Es_Jefe"] = (to_int(df["tip_parentesco"]) == 1).astype("Int64")

```

Listing 11: Indicador binario de jefe de hogar

Genera un 1 si la persona es el jefe o jefa del hogar (`tip_parentesco == 1`) y un 0 en caso contrario. Este indicador es fundamental para muchas métricas que deben calcularse a nivel de *hogar* (no de persona): número de hogares, gasto del hogar, tamaño del hogar, etc. En DAX, esto se filtraba con `CALCULATE([Metros], tabla[tip_parentesco] = 1)`.

4.4.11. Columnas 15–16: Sexo_Jefe y Sexo_Jefe_Label (líneas 218–225)

```

218 _jefe_mask = (to_int(df["tip_parentesco"]) == 1)
219 df["Sexo_Jefe"] = pd.array([pd.NA] * len(df), dtype="Int64")
220 df.loc[_jefe_mask, "Sexo_Jefe"] = to_int(df.loc[_jefe_mask, "
    sexo_persona"])

```

Listing 12: Sexo del jefe del hogar

Para los no-jefes del hogar, `Sexo_Jefe` queda como nulo (`pd.NA`). Solo para la fila del jefe de hogar, copio el valor de `sexo_persona`. Esto me permite después filtrar por sexo del jefe sin mezclar el sexo de los demás miembros del hogar.

4.4.12. Columna 17: `Personas_por_Cuarto` (líneas 227–230)

```
227 _personas = pd.to_numeric(df["num_personas_hogar"], errors="coerce")
228 _cuartos  = pd.to_numeric(df["num_cuartos_vivienda"], errors="coerce").
      replace(0, np.nan)
229 df["Personas_por_Cuarto"] = (_personas / _cuartos).astype("Float64")
```

Listing 13: Densidad de personas por cuarto

Este es el cálculo del **índice de hacinamiento**. Divido el número de personas del hogar entre el número de cuartos de la vivienda. Reemplazo el 0 en el denominador por NaN para evitar una división por cero (que generaría un valor infinito y rompería el modelo). El resultado es un número de punto flotante (tipo `Float64`).

4.4.13. Columna 18: `Cat_Hacinamiento` (líneas 232–237)

Clasifica el resultado del cálculo anterior:

- **Hacinado:** Más de 3 personas por cuarto.
- **No hacinado:** 3 personas o menos por cuarto.
- **Sin información:** El cálculo resultó nulo.

El umbral de 3 personas por cuarto es el estándar oficial colombiano para determinar hacinamiento crítico (uno de los 15 indicadores del IPM, específicamente el **I15**).

4.4.14. Columna 19: `Material del Piso` (líneas 240–248)

Traduce `tip_mat_pisos` a texto:

Código	Etiqueta
1	Alfombra o tapete, mármol, parqué, madera pulida y lacada
2	Baldosa, vinilo, tableta, ladrillo
3	Cemento, gravilla
4	Madera burda, madera en mal estado, tabla, tablón
5	Tierra o arena
6	Otro
Cualquier otro	Sin información

4.4.15. Columna 20: Orden_Paredes (línea 251)

Una copia del valor numérico de `tip_mat_paredes` para usarlo como eje de ordenamiento en los gráficos de barras. Lo mantengo separado de la versión texto.

4.4.16. Columna 21: Tipo_Ocupacion_txt (líneas 253–258)

Traduce el código de tenencia de la vivienda: arrendatario, propietario pagando, propietario total, con permiso o poseedor sin título.

4.4.17. Columna 22: Rango_Edad (líneas 260–274)

```
260 _edad = pd.to_numeric(df["Edad"], errors="coerce")
261 _rango_cond = [
262     _edad <= 4, _edad <= 9, _edad <= 14, _edad <= 19, _edad <= 24,
263     _edad <= 29, _edad <= 34, _edad <= 39, _edad <= 44, _edad <= 49,
264     _edad <= 54, _edad <= 59, _edad <= 64, _edad <= 69, _edad <= 74,
265     _edad <= 79, _edad >= 80,
266 ]
267 _rango_labels = [
268     "00-04", "05-09", "10-14", "15-19", "20-24",
269     "25-29", "30-34", "35-39", "40-44", "45-49",
270     "50-54", "55-59", "60-64", "65-69", "70-74",
271     "75-79", "80+",
272 ]
273 df["Rango_Edad"] = np.select(_rango_cond, _rango_labels, default=pd.NA)
```

Listing 14: Agrupación de edad en rangos quinquenales

Esta transformación agrupa cada edad individual en un rango quinquenal (de 5 en 5 años). Es el estándar demográfico para crear las pirámides poblacionales. Uso `np.select` porque es la versión vectorizada de un `SWITCH(TRUE(), ...)` de DAX: evalúa cada condición en orden y asigna la primera etiqueta cuya condición sea verdadera.

4.4.18. Columna 23: Orden_Edad (líneas 277–283)

Asigna un número de 1 a 17 a cada rango de edad para garantizar el orden correcto en la pirámide poblacional. Sin esto, Power BI ordenaría los rangos alfabéticamente ("00-04" iría antes que "10-14" pero después de "05-09", lo cual casualmente es correcto en texto, pero no es confiable para todos los casos).

4.4.19. Columna 24: Regimen_Salud_Limpio (líneas 285–288)

Traduce el código de régimen de salud:

- 0: Ninguno, 1: Contributivo, 2: Especial, 3: Subsidiado, 9: No sabe.

5. Procesamiento por Chunks: El Motor del ETL

5.1. Inicialización del lector de Parquet (líneas 310–324)

```
310 pf = pq.ParquetFile(PARQUET_IN)
311 total_rows = pf.metadata.num_rows
312 num_row_groups = pf.metadata.num_row_groups
```

Listing 15: Apertura del archivo fuente y lectura de metadatos

`pq.ParquetFile` no carga el archivo en memoria. Solo abre un *puntero* al archivo y lee sus metadatos del encabezado: cuántas filas tiene en total y en cuántos grupos de filas está dividido. Los metadatos del formato Parquet están en el pie del archivo y ocupan kilobytes, por lo que esta operación es instantánea independientemente del tamaño del archivo.

5.2. El ciclo principal (líneas 326–368)

```
326 for batch in pf.iter_batches(batch_size=CHUNK_ROWS):
327     chunk_num += 1
328
329     # Convertir chunk a pandas
330     df_chunk = batch.to_pandas()
331
332     # Aplicar todas las transformaciones
333     df_chunk = calcular_columnas(df_chunk)
334
335     # Ordenar columnas: base primero, calculadas al final
336     cols_base = [c for c in df_chunk.columns if c not in COLS_CALCULADAS]
337     cols_finales = cols_base + [c for c in COLS_CALCULADAS if c in
338 df_chunk.columns]
339     df_chunk = df_chunk[cols_finales]
340
341     # Convertir a PyArrow table
342     table = pa.Table.from_pandas(df_chunk, preserve_index=False)
343
344     # Inicializar escritor con el schema del PRIMER chunk
345     if writer is None:
346         schema = table.schema
347         writer = pq.ParquetWriter(PARQUET_OUT, schema, compression="
348 snappy")
349
350     # Escribir chunk al archivo
351     writer.write_table(table)
352
353     # Liberar memoria
354     del df_chunk, table, batch
355     gc.collect()
```

Listing 16: Iteración por lotes y escritura incremental

Hay varios aspectos técnicos críticos en este bloque:

1. **El ParquetWriter se inicializa solo con el primer chunk.** Esto es intencional: el *schema* (estructura de columnas y tipos) se define a partir del primer lote procesado y se fija para todo el archivo. Todos los lotes subsiguientes deben entregar exactamente las mismas columnas con los mismos tipos. Si hubiera inconsistencias entre lotes (por ejemplo, baches de datos donde una columna aparece vacía en un lote y con datos en otro), el *schema* del primer lote actúa como contrato que fuerza la coherencia.
2. **El ordenamiento de columnas.** Separo las columnas en *base* (las que venían originalmente) y *calculadas* (las que acabo de añadir), y las pongo en ese orden. Esto hace que el parquet resultante sea fácil de explorar: las primeras N columnas son los datos originales del DNP; las últimas 24 son mis columnas analíticas.
3. **El `del` y `gc.collect()`.** Al final de cada iteración, elimino explícitamente las variables del chunk y llamo al recolector de basura de Python. Sin esto, Python esperarí a que su recolector automático decidiera liberar la memoria, lo que puede demorar varias iteraciones y causar que la memoria acumulada supere el límite del sistema.
4. **Compresión Snappy.** El parámetro `compression="snappy"` activa el algoritmo de compresión Snappy de Google durante la escritura. Este algoritmo es ideal para análisis de datos porque prioriza la *velocidad de lectura* sobre la tasa de compresión. El resultado es que Power BI puede leer el Parquet con Snappy casi tan rápido como si fuera un archivo de datos en binario puro, y al mismo tiempo el archivo ocupa entre 3 y 5 veces menos espacio en disco que el equivalente CSV.

6. Reemplazo Atómico y Verificación Final

6.1. Reemplazo del archivo original (líneas 380–384)

```
380 if os.path.exists(PARQUET_FINAL):  
381     os.remove(PARQUET_FINAL)  
382 os.rename(PARQUET_OUT, PARQUET_FINAL)
```

Listing 17: Reemplazo atómico del archivo definitivo

Solo si el proceso completó todos los chunks sin errores llego a esta sección. Primero elimino el archivo original y luego renombro el temporal al nombre definitivo. Este es el patrón de **reemplazo atómico**: si el proceso falla en cualquier punto antes de llegar aquí, el archivo original permanece intacto.

6.2. Verificación automática de tipos (líneas 403–441)

```

403 pf_check = pq.ParquetFile(PARQUET_FINAL)
404 df_sample = next(pf_check.iter_batches(batch_size=50_000)).to_pandas()

```

Listing 18: Verificación automática del Parquet generado

Diseñé una verificación automática al final: el script lee las primeras 50.000 filas del Parquet *que acaba de generar* y comprueba que cada columna crítica tiene el tipo de dato esperado. Si algo está mal (por ejemplo, si `Es_Jefe` quedó como `object` en lugar de `Int64`), el script lo reporta con un símbolo antes de terminar. Si todo está correcto, imprime para cada columna.

7. Cómo Ejecutar el Script

7.1. Pasos de ejecución

1. Abrir una terminal (PowerShell o CMD) en el computador.
2. Verificar que el archivo fuente existe en la ruta configurada: `D:\VVANEGASA\My Documents\sisben_master.parquet`.
3. Ejecutar el script:

```

1 python "D:\VVANEGASA\Desktop\antigravity_sisben\
   etl_sisben_definitivo.py"
2

```

4. El script imprimirá el progreso en tiempo real, indicando el número de lote, las filas procesadas, el tiempo transcurrido y el tiempo estimado restante (ETA).
5. Al terminar, aparecerá el resumen final con el número total de filas, el tamaño del archivo resultante y el tiempo total.

7.2. ¿Cuándo se debe volver a ejecutar?

El ETL debe volver a ejecutarse en los siguientes casos:

- El DNP entregó una nueva versión de la base de datos municipal.
- Se requiere agregar una nueva columna calculada porque el tablero necesita un nuevo análisis.
- El archivo `sisben_master.parquet` se dañó o fue sobrescrito accidentalmente.

8. Integración con Power BI

Una vez generado el Parquet, Power BI lo lee mediante la siguiente expresión de Power Query:

```
1 = Parquet.Document(File.Contents(  
2     "D:\VVANEGASA\My Documents\sisben_master_final.parquet"  
3 ))
```

Listing 19: Expresión Power Query para leer el Parquet

El modelo de Power BI **no necesita transformar nada** al leer este Parquet: todas las columnas calculadas ya están pre-procesadas, con sus tipos correctos y sus etiquetas de texto listas. Esto reduce el tiempo de apertura del archivo de Power BI de más de 20 minutos (con columnas DAX calculadas en vivo) a menos de 3 minutos.

9. Resumen de Columnas Generadas por el ETL

Columna generada	Tipo	Descripción
orden_clasificacion	Int64	Orden numérico del grupo Sisbén (A=1, B=2, C=3, D=4)
Tipo_Vivienda_txt	String	Etiqueta oficial del tipo de vivienda
Material de las Paredes	String	Etiqueta oficial del material de paredes
Energia_txt	String	Tenencia de energía eléctrica en texto
Acueducto_txt	String	Tenencia de acueducto en texto
Alcantarillado_txt	String	Tenencia de alcantarillado en texto
Gas_txt	String	Tenencia de gas domiciliario en texto
Recoleccion_txt	String	Tenencia de recolección de basuras en texto
Clase_txt	String	Zona urbano/rural en texto
Sanitario_txt	String	Tipo de unidad sanitaria en texto
Privaciones_Vivienda	Int64	Suma de indicadores IPM de vivienda (I11+I12+I13+I14+I15)
Nivel_Calidad_Vivienda	String	Sin privaciones / Moderada / Alta
Zona	String	Alias de Clase_txt
Es_Jefe	Int64	1 si es jefe del hogar, 0 si no
Sexo_Jefe	Int64	Sexo del jefe (solo en la fila del jefe, nulo en otros)
Sexo_Jefe_Label	String	Hombre jefe / Mujer jefe
Personas_por_Cuarto	Float64	Índice de hacinamiento del hogar
Cat_Hacinamiento	String	Hacinado / No hacinado / Sin información

Columna generada	Tipo	Descripción
Material del Piso	String	Etiqueta oficial del material de piso
Orden_Paredes	Int64	Código numérico de paredes para ordenamiento
Tipo_Ocupacion_txt	String	Etiqueta oficial de tenencia de vivienda
Rango_Edad	String	Rango quinquenal: 00-04, 05-09, ..., 80+
Orden_Edad	Int64	Orden numérico del rango de edad (1 a 17)
Regimen_Salud_Limpio	String	Régimen de salud en texto (Subsidiado, Contributivo, etc.)

10. Notas Técnicas y Advertencias

IMPORTANTE — Columna estado y las Matrices DNP. La columna `estado` permanece en formato texto (String) en el Parquet. Esto es intencional. Las fórmulas DAX de las Matrices 1, 2 y 3 del tablero de Power BI usan comparaciones de texto (= "0", IN {"60", "63"}). Si alguien convierte esta columna a entero en el ETL, deberá corregir también todas las medidas DAX del tablero para que usen comparaciones numéricas (= 0, IN {60, 63}).

RENDIMIENTO. El parámetro `CHUNK_ROWS = 150_000` fue calibrado para un computador con 16 GB de RAM. Si el equipo donde se ejecute tiene 8 GB, se recomienda reducirlo a 100_000. Si tiene 32 GB o más, se puede aumentar a 300_000 para acelerar el proceso. El resultado final del Parquet es idéntico en todos los casos.